Software Architecture & Design Specification

for

Credit Card Fraud Detection and Model Performance Evaluation System

Version 1.1 (SADS-CCFD-1.1)

Prepared by: Team Error 404

Date: 12-Oct-2026

Derived from: SRS-CCFD-1.2 · Verified by: STP-CCFD-1.1

Table of Contents

- Revision History
- 1. Introduction
  - 1.1 Purpose
  - 1.2 Scope
  - 1.3 References
  - 1.4 Overview
  - 1.5 Document Conventions
- 2. Architecture
  - 2.1 Component Diagram and Description
  - 2.2 Architectural Pattern
  - 2.3 Traceability to Requirements
  - 2.4 Security Architecture
  - 2.5 Deployment View
  - 2.6 Data Design
- 3. Design
  - 3.1 Sequence Diagrams
  - 3.2 API Design
  - 3.3 Error Handling
  - 3.4 Module Breakdown
  - 3.5 User Interface Design Notes
- Appendix A: Glossary
- Project Team

Revision History

Name	Date	Reason for Changes	Version
Team Error 404	11-Oct-2026	Initial architecture and design specification for Release 1.0, derived from SRS v1.1.	1.0
Team Error 404	12-Oct-2026	Consistency review with SRS v1.2 and STP v1.1: one component diagram with named interfaces (IRestApi, IPredict, IDatasetStore); microservices read the dataset from the store by dataset_id and the Job Manager calls POST /predict synchronously in a background worker (no microservice polling); API key required on all REST endpoints; security controls renumbered 1–6 with correct STP test-case IDs; API table rebuilt with complete request/response contracts; job failures reported as job reasons instead of HTTP 502/504; redrawn sequence diagrams with failure paths; merged the duplicate traceability tables into Section 2.3.	1.1

1. Introduction

1.1 Purpose

This document describes the software architecture and detailed design of Release 1.0 of the Credit Card Fraud Detection and Model Performance Evaluation System. It defines the system's components and their interfaces, the architectural pattern and its rationale, the security architecture, the deployment topology, the data structures, the key interaction flows (sequence diagrams), the REST API design, error handling, and the module breakdown that developers implement against. It is a proposed design: no implementation exists yet, and the module and file names in Section 3.4 are the planned structure.

1.2 Scope

In scope: the Web Client, the Flask Web Server (routes, authentication, validation, job management, storage), the three model microservices (ML, DL, QML), the dataset store, and all interactions between them. Out of scope (per SRS v1.2): model retraining pipelines, deployment beyond a single host (TBD-01), public-internet hardening and TLS (TBD-02), and physical quantum hardware.

1.3 References

1. Software Requirements Specification, Credit Card Fraud Detection and Model Performance Evaluation System, v1.2 (SRS-CCFD-1.2) — the requirements this design realizes.
2. Software Test Plan, same system, v1.1 (STP-CCFD-1.1) — the test cases that verify this design.
3. IEEE Std 1016-2009, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions* — content reference for this document.
4. SE Mini-Project Deliverables Part-1 — course guidelines (architecture: component diagram and description, architectural pattern, traceability to requirements, security architecture; design: at least two UML sequence diagrams, API design, error handling).

1.4 Overview

Section 2 presents the architecture: the component diagram (2.1), the pattern and rationale (2.2), traceability to requirements (2.3), the security architecture (2.4), deployment (2.5) and data design (2.6). Section 3 presents the detailed design: sequence diagrams (3.1), API design (3.2), error handling and the job lifecycle (3.3), the module breakdown (3.4) and user interface notes (3.5).

1.5 Document Conventions

Diagrams use UML notation: components carry the «component» stereotype, a circle (ball) is a provided interface, a half-circle (socket) is a required interface, and dashed «use» arrows are dependencies; sequence diagrams use alt, par and loop combined fragments. Requirement identifiers (REQ-FD-nn, REQ-UI-nn, REQ-API-nn, NFR-nn, NFR-Snn, NFR-Qnn, SO-nn, SR-nn, BR-nn) refer to SRS v1.2; test-case identifiers (TC-XXX-nn) refer to STP v1.1.

2. Architecture

2.1 Component Diagram and Description

Figure 1 shows the components of the system and the three interfaces through which they interact. The Web Client talks only to the Web Server (IRestApi); the Web Server talks to the model microservices only through IPredict; the dataset store is reached through IDatasetStore, read/write by the Web Server and read-only by the microservices.

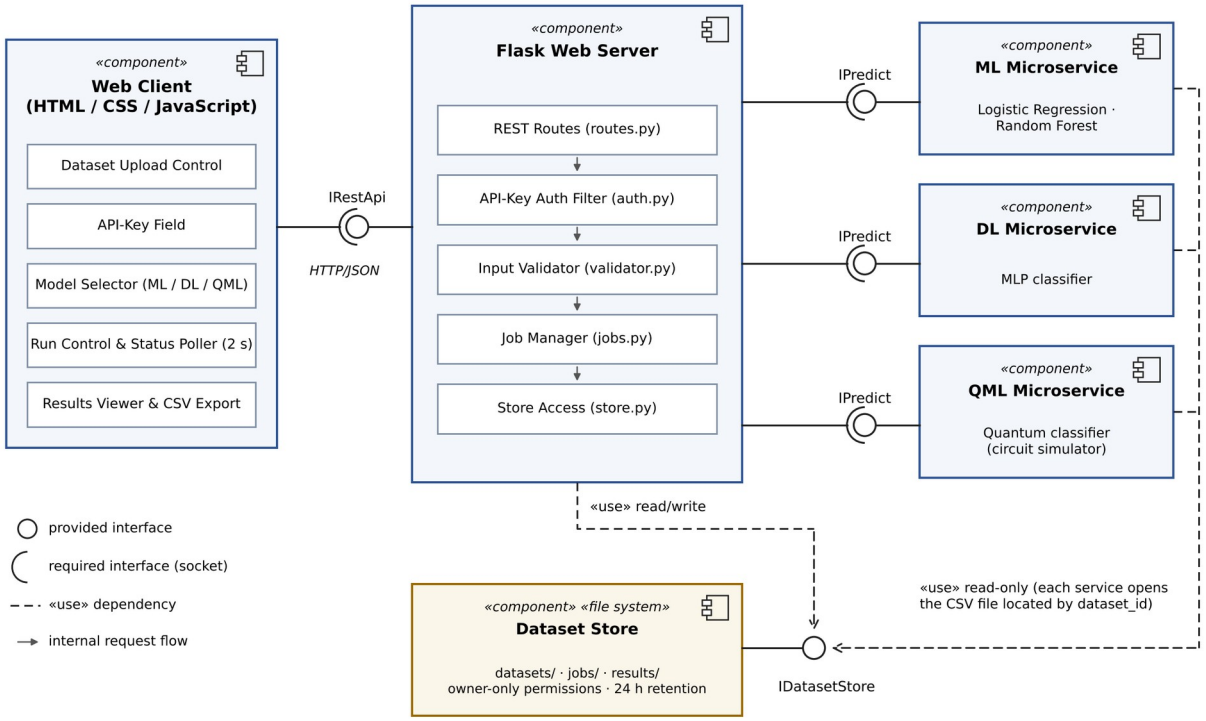


Figure 1. UML component diagram.

Component	Responsibility	Provided interface	Required interface
Web Client	API-key field, dataset upload, model selection, run control with 2-second status polling, results table (50 rows/page), metrics and confusion matrix, CSV export generated in the browser	Browser UI (used by the General User)	IRestApi
Flask Web Server	REST routing, API-key authentication, input validation, asynchronous job management, storage and retention; contains no model logic	IRestApi	IPredict (selected microservice); IDatasetStore (read/write)
— REST Routes	Maps the three endpoints to handlers; serializes JSON responses; global error handler	(internal)	Auth Filter, Input Validator, Job Manager, Store Access
— API-Key Auth Filter	Rejects requests with a missing or invalid key (HTTP 401) before any processing	(internal)	—
— Input Validator	File-level checks (type, size, delimiter, required columns), row-level checks (Class ∈ {0,1}, numeric features), strict JSON schema validation	(internal)	—
— Job Manager	Creates job records; runs each job in a background worker that calls POST /predict with the dataset_id and waits up to 600 s; records completed or failed with a reason	(internal)	IPredict; Store Access

Component	Responsibility	Provided interface	Required interface
— Store Access	Saves datasets, job records and results; enforces owner-only permissions; deletes data when the 24-hour retention period ends	(internal)	IDatasetStore
ML Microservice	Hosts trained Logistic Regression and Random Forest classifiers; runs the one named in its configuration	IPredict (POST /predict, GET /health)	IDatasetStore (read-only)
DL Microservice	Hosts the trained multilayer perceptron	IPredict	IDatasetStore (read-only)
QML Microservice	Hosts the quantum classifier executed on a circuit simulator	IPredict	IDatasetStore (read-only)
Dataset Store	Server-local directory: datasets/, jobs/, results/	IDatasetStore	—

Interface	Provided by	Required by	Operations
IRestApi	Flask Web Server	Web Client, developer tools	POST /api/datasets; POST /api/analyses; GET /api/analyses/{job_id} (Section 3.2)
IPredict	Each model microservice	Job Manager	POST /predict {dataset_id}; GET /health (Section 3.2)
IDatasetStore	Dataset Store (file system)	Store Access (read/write); microservices (read-only)	Save/read/delete datasets/<dataset_id>.csv, jobs/<job_id>.json, results/<job_id>.json

2.2 Architectural Pattern

The system follows a **three-tier client–server pattern** (presentation → application → model) combined with **microservices** for the model tier and the **asynchronous job pattern** for long-running analyses:

- **Presentation tier:** the Web Client (browser, HTML/CSS/JS, no build toolchain).
- **Application tier:** the Flask Web Server, which owns request handling, authentication, validation, storage and orchestration but no model logic.
- **Model tier:** three independently deployable microservices behind the uniform POST /predict / GET /health contract.

Rationale.

1. **Separation of concerns:** UI, orchestration and model execution evolve independently; a new model is added as a microservice and a configuration entry, without changing the client (SRS REQ-FD-10, NFR-Q02, Section 6 reuse objective).
2. **Independent testability:** each microservice is verifiable in isolation against its HTTP contract (SRS NFR-Q05; STP TC-MOD-06).
3. **Responsiveness:** model runs vary in duration with data size and model type, so the asynchronous job pattern with 2-second status polling keeps the UI responsive regardless of run length (SRS REQ-FD-12, NFR-02, NFR-04).
4. **Team fit:** the pattern maps cleanly onto parallel development by a small team.

Passing data by reference. Datasets can be up to 500 MB, so they are never copied into HTTP request bodies between components. The Web Server stores the file once and passes only the dataset_id; the

selected microservice opens the file from the dataset store. This keeps IPredict small and identical for all three services, and it is the reason Release 1.0 runs all processes on one host (Section 2.5).

Alternative considered. A monolithic Flask application embedding all three models was rejected: it would couple model dependencies (including the QML simulator stack) into one process, remove independent deployability (SRS NFR-06), and make per-model failure isolation harder (SRS REQ-FD-14).

2.3 Traceability to Requirements

Design element	SRS requirements realized	Verified by (STP)
Web Client (index.html, app.js, style.css)	REQ-FD-01, REQ-FD-06, REQ-FD-18 – REQ-FD-21, REQ-UI-01 – REQ-UI-06, NFR-04, NFR-S02, BR-02, BR-05	TC-RES-01 – TC-RES-07, TC-MOD-01, TC-MET-03, TC-JOB-02, TC-PERF-04, TC-SAFE-01, TC-SAFE-04
REST Routes (routes.py)	REQ-API-01 – REQ-API-05	TC-API-01 – TC-API-05
API-Key Auth Filter (auth.py)	SR-03	TC-SEC-01 – TC-SEC-03, TC-SEC-10
Input Validator (validator.py)	REQ-FD-02 – REQ-FD-04, SR-01, SR-02, BR-01, NFR-01, NFR-03, NFR-07	TC-UPL-01 – TC-UPL-08, TC-SEC-04 – TC-SEC-06, TC-PERF-01, TC-PERF-03, TC-PERF-06
Job Manager (jobs.py)	REQ-FD-11, REQ-FD-12, REQ-FD-14, NFR-05, NFR-06, BR-04	TC-MOD-02 – TC-MOD-04, TC-JOB-01 – TC-JOB-06, TC-PERF-05
Store Access and Dataset Store (store.py)	REQ-FD-05, REQ-FD-22, SR-05	TC-UPL-01, TC-JOB-04, TC-SEC-09
ML / DL / QML microservices	REQ-FD-07 – REQ-FD-10, REQ-FD-13, REQ-FD-15 – REQ-FD-17, NFR-Q05, BR-03	TC-MOD-02 – TC-MOD-06, TC-RES-03, TC-MET-01 – TC-MET-05
Error taxonomy and global error handler (Section 3.3)	REQ-API-04, SR-06, SR-04 (no data in logs or messages)	TC-API-04, TC-API-05, TC-SEC-07, TC-SEC-08
Asynchronous job + polling design (SD-02)	REQ-FD-12, REQ-FD-20, NFR-02	TC-JOB-02, TC-RES-04, TC-PERF-02
Analytical-only design (no transaction endpoint, notice in results)	NFR-S01, NFR-S02, BR-05	TC-SAFE-01, TC-SAFE-02
Uniform IPredict contract	NFR-Q02	Design inspection; TC-MOD-06

2.4 Security Architecture

Security is layered so that each layer independently upholds the security objectives (SO-01 confidentiality, SO-02 authorized invocation, SO-03 input trustworthiness):

1. **Edge authentication (SO-01, SO-02 → SR-03).** The API-Key Auth Filter runs first on every REST route, including GET /api/analyses/{job_id}. The key is sent as `Authorization: Bearer <key>`; the server compares it with the key held in its configuration (an environment variable, never in source code) using a constant-time comparison. A missing or invalid key yields HTTP 401 with no side effects. Job identifiers are random UUIDv4 values. (STP TC-SEC-01, TC-SEC-02, TC-SEC-03, TC-SEC-10.)
2. **Input validation before processing (SO-01, SO-03 → SR-01, SR-02).** The Input Validator enforces file type, size and schema and strict JSON schemas before the Job Manager or any microservice is

involved; a rejected file is discarded, never stored, and never reaches model code. (TC-SEC-04, TC-SEC-05, TC-SEC-06, TC-UPL-02 – TC-UPL-07.)

3. **Data minimization in logs and display (SO-01 → SR-04).** Logs record request metadata (endpoint, status, dataset_id, job_id, durations) but never field values or payloads. The Web Client and the CSV export show only row index, predicted class and probability plus aggregate metrics, so no other dataset column needs masking; validation messages name columns and row numbers but never echo cell values. (TC-SEC-08.)
4. **Protected storage and retention (SO-01 → SR-05).** The dataset store is a dedicated directory with owner-only permissions (0700 directories, 0600 files on Linux; an equivalent ACL on Windows) for the service account that runs all four processes. A purge task in store.py runs every 10 minutes and deletes each dataset, its job records and its results once expires_at (upload + 24 h) has passed. (TC-SEC-09.)
5. **Safe failure (SO-01, SO-02 → SR-06).** The global error handler (Section 3.3) returns only a machine-readable code and a human-readable message — never stack traces, paths or configuration; details are logged server-side only. (TC-SEC-07.)
6. **Service isolation (SO-03 → REQ-FD-14).** The microservices listen only on the loopback interface, so they cannot be reached from the Web Client or the LAN. A crashing or timing-out microservice fails its own job with a recorded reason; the Job Manager never fabricates results. (TC-JOB-05, TC-JOB-06; BR-04.)

Out of scope for Release 1.0 (SRS TBD-02): TLS termination and secret management for access from other machines.

2.5 Deployment View

Release 1.0 targets a single host. Each component runs as an independent OS process under one dedicated service account:

Process	Port	Listens on	Notes
Flask Web Server	5000	Host address (or localhost)	Serves the Web Client static files and the REST API; owns the dataset store
ML microservice	5001	127.0.0.1 only	POST /predict, GET /health
DL microservice	5002	127.0.0.1 only	POST /predict, GET /health
QML microservice	5003	127.0.0.1 only	POST /predict, GET /health; needs simulator libraries

The Web Client is served as static files by the Flask server and runs in the user's browser. All inter-process communication is HTTP/JSON over TCP/IP. The dataset store path is set in one shared configuration value read by all four processes; the microservices need read access only. Because each microservice is an independent process, stopping one does not prevent the others from serving (SRS NFR-06). Running the microservices on other machines would require a shared file store and is deferred (SRS TBD-01).

2.6 Data Design

- **Dataset CSV (input).** UTF-8, comma-delimited, header row; required `Class` column (0/1); the feature columns expected by the trained models (documented schema), all numeric; extra columns ignored; ≤ 500 MB and ≤ 1,000,000 rows (SRS REQ-FD-02, REQ-FD-03). Stored as `datasets/<dataset_id>.csv` with a sidecar `datasets/<dataset_id>.json` = `{ dataset_id, rows, uploaded_at, expires_at }`.

- **Job record.** `jobs/<job_id>.json` = `{ job_id, dataset_id, model_type, status: queued | running | completed | failed, created_at, started_at?, finished_at?, reason?: { code, message } }`.
- **Result.** `results/<job_id>.json` = `{ model_name, predictions: [ { row, class, probability } ], metrics: { accuracy, precision, recall, f1, roc_auc, confusion_matrix: { tp, tn, fp, fn } } }`, served through GET /api/analyses/{job_id} and rendered by the client.
- **Identifiers.** dataset_id and job_id are random UUIDv4 strings generated by the Web Server.
- **Retention.** expires_at = uploaded_at + 24 h; the purge task deletes the dataset, its sidecar, and every job record and result that references it (SRS SR-05).
- No database is required; the file-based dataset store is the system of record for Release 1.0 (SRS Section 6).

3. Design

3.1 Sequence Diagrams

SD-01 — Dataset upload and validation

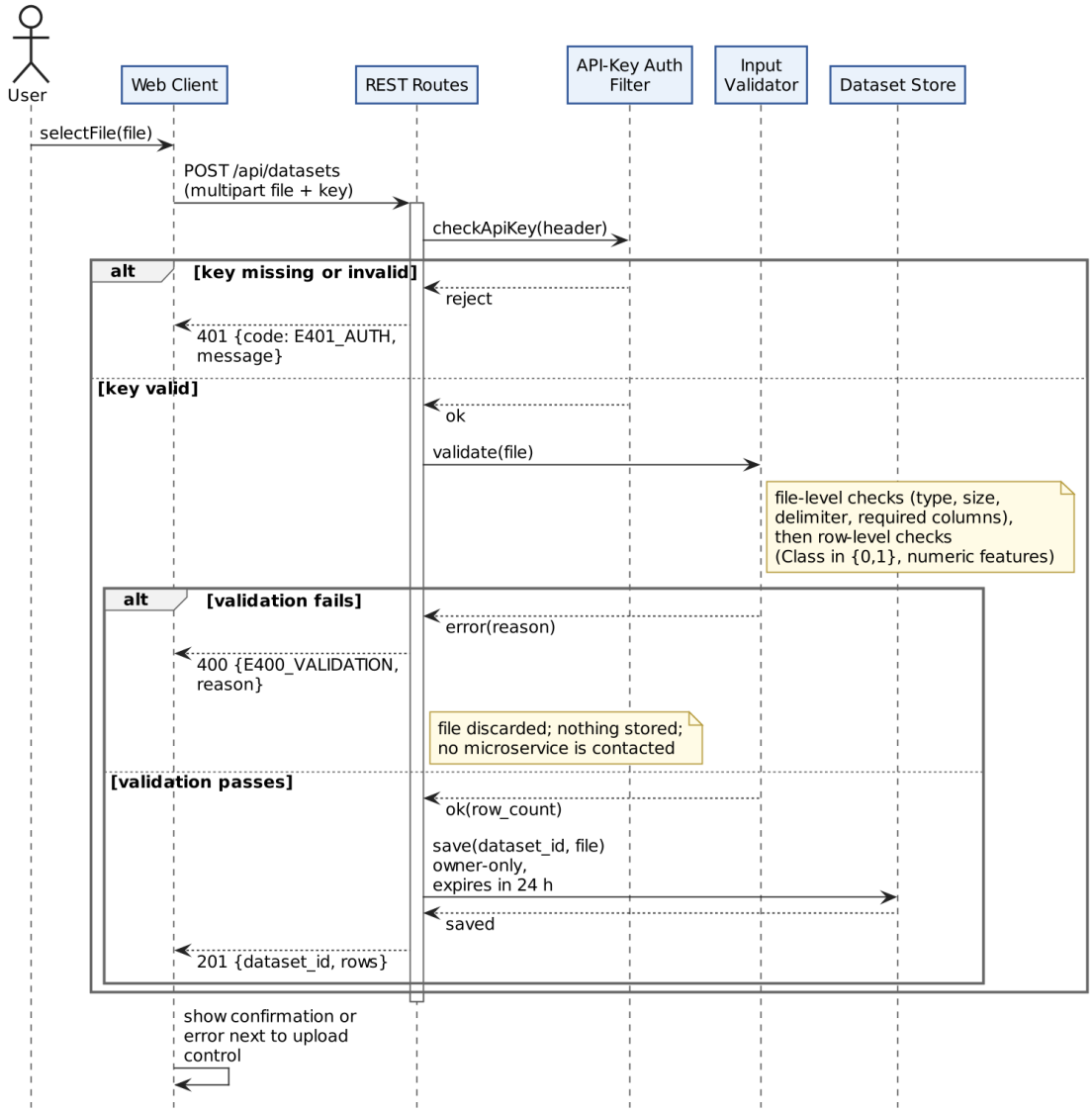


Figure 2. UML sequence diagram — dataset upload and validation.

The user selects a file in the Web Client, which submits it via POST /api/datasets with the API key. The REST route first calls the API-Key Auth Filter; a missing or invalid key ends the request with HTTP 401. Otherwise the Input Validator runs the file-level checks (type, size, delimiter, required columns) and then the row-level checks. If validation fails, the server returns HTTP 400 naming the reason and the file is discarded: nothing is stored and no microservice is contacted. If it passes, the file is saved to the dataset store with an expiry 24 hours ahead and the server returns HTTP 201 with the dataset_id and row count. The client shows the confirmation or the error next to the upload control. This realizes SRS REQ-FD-01 – REQ-FD-05, REQ-API-01, SR-01, SR-03 and SR-05, within the timing limits of NFR-01, NFR-03 and NFR-07.

SD-02 — Fraud-detection run (asynchronous)

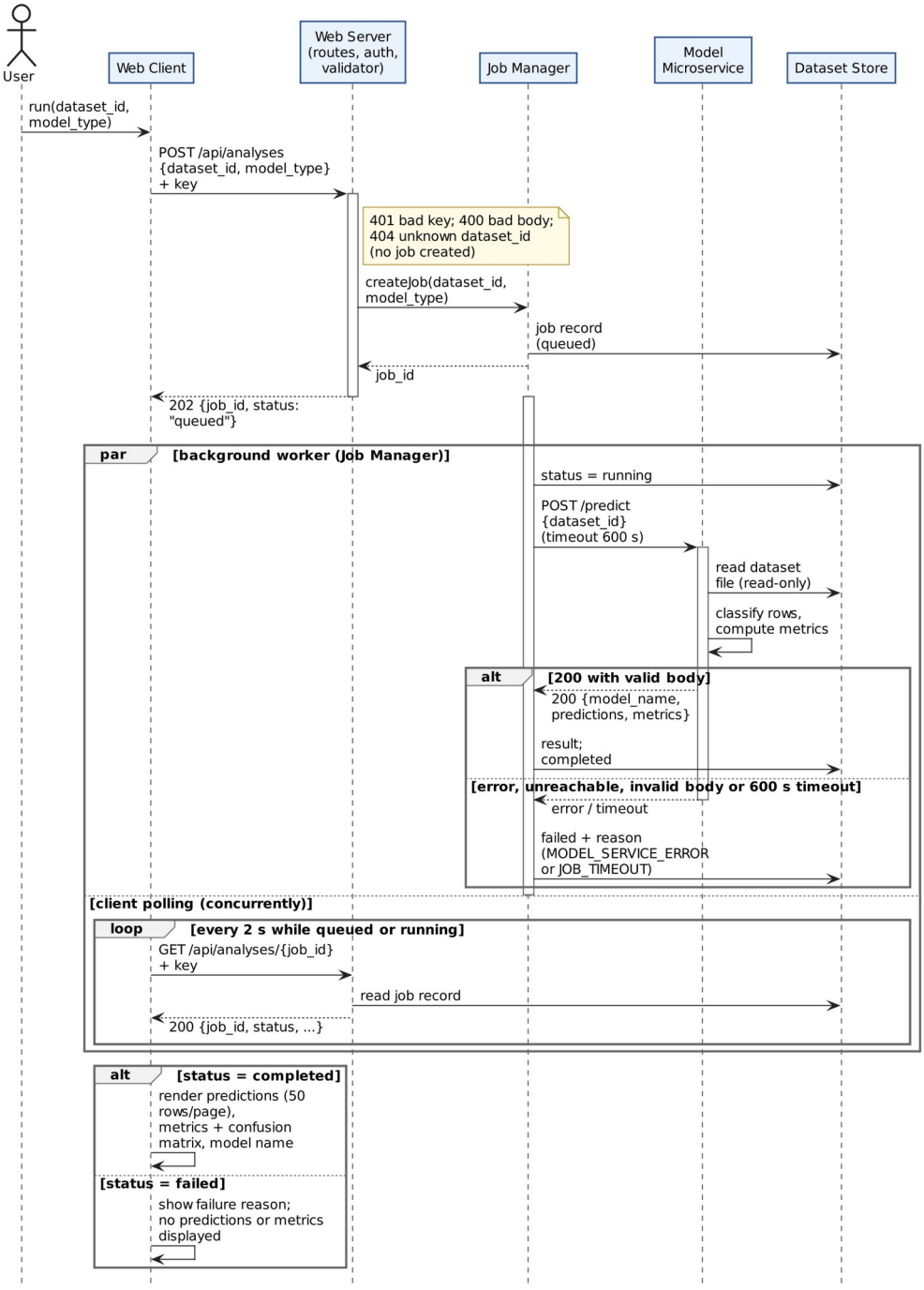


Figure 3. UML sequence diagram — asynchronous fraud-detection run.

The user starts a run for an accepted dataset and a chosen model type. After authentication and request validation (401, 400, or 404 for an unknown dataset_id, with no job created), the server creates a job record in the queued state and immediately returns HTTP 202 with the job_id. Two activities then proceed in parallel (par fragment). In the background worker, the Job Manager marks the job running and calls POST /predict on the selected microservice with only the dataset_id, waiting at most 600 seconds; the microservice reads the CSV from the dataset store, classifies every row and computes the metrics. A valid 200 response is stored and the job is marked completed; an error response, connection failure, invalid body or timeout marks the job failed with reason MODEL_SERVICE_ERROR or JOB_TIMEOUT. Meanwhile, the client polls GET /api/analyses/{job_id} every 2 seconds. Once a terminal state is reported, polling stops and the client shows either the results or the failure reason. This realizes SRS REQ-FD-10 – REQ-FD-17, REQ-FD-20, REQ-API-02, REQ-API-03 and BR-04.

3.2 API Design

All REST endpoints require `Authorization: Bearer <api-key>` (SR-03) and return JSON with `Content-Type: application/json`. Microservice endpoints are reachable only from the local host and need no key.

Method & path	Auth	Success response	Error responses
POST /api/datasets	API key	201 {dataset_id, rows}	400 E400_VALIDATION; 401 E401_AUTH; 405; 500
POST /api/analyses	API key	202 {job_id, status: "queued"}	400 E400_VALIDATION; 401 E401_AUTH; 404 E404_NOT_FOUND (dataset); 405; 500
GET /api/analyses/{job_id}	API key	200 job object (below)	401 E401_AUTH; 404 E404_NOT_FOUND (job); 405; 500
POST /predict (microservice)	Local host only	200 {model_name, predictions, metrics}	400 E400_VALIDATION (bad body); 404 E404_NOT_FOUND (dataset); 500 E500_MODEL
GET /health (microservice)	Local host only	200 {"status": "ok"}	503 {"status": "not_ready"}

POST /api/datasets — request: multipart/form-data with one part named `file` containing the CSV.

```
HTTP/1.1 201 Created
{ "dataset_id": "3f1c9a52-7d0e-4b8f-9a61-2c5e8d7b1f04", "rows": 1000 }
```

POST /api/analyses — request body (no other fields allowed; model_type must be one of "ml", "dl", "qml"):

```
{ "dataset_id": "3f1c9a52-7d0e-4b8f-9a61-2c5e8d7b1f04", "model_type": "ml" }

HTTP/1.1 202 Accepted
{ "job_id": "a8e2d6c4-1b3f-4e5a-8c7d-9f0e1a2b3c4d", "status": "queued" }
```

GET /api/analyses/{job_id} — response while queued or running, when completed, and when failed:

```
200 { "job_id": "...", "dataset_id": "...", "model_type": "ml", "status": "running" }

200 { "job_id": "...", "dataset_id": "...", "model_type": "ml", "status": "completed",
      "model_name": "random_forest",
      "predictions": [ { "row": 0, "class": 0, "probability": 0.0132 }, ... ],
      "metrics": { "accuracy": 0.9930, "precision": 0.8125, "recall": 0.7647,
                   "f1": 0.7879, "roc_auc": 0.9412,
                   "confusion_matrix": { "tp": 13, "tn": 980, "fp": 3, "fn": 4 } } }

200 { "job_id": "...", "dataset_id": "...", "model_type": "qml", "status": "failed",
      "reason": { "code": "MODEL_SERVICE_ERROR",
                  "message": "The model service is unavailable; the job was marked
failed." } }
```

(The metric values above illustrate the format only.)

POST /predict (microservice) — request `{ "dataset_id": "<id>" }`; response `{ "model_name", "predictions": [ { "row", "class", "probability" } ], "metrics": { "accuracy", "precision", "recall", "f1", "roc_auc", "confusion_matrix": { "tp", "tn", "fp", "fn" } } }`, with metrics rounded to four decimal places (REQ-FD-15).

Error body (every 4xx/5xx response from the Web Server and from POST /predict):

```
{ "code": "E400_VALIDATION", "message": "missing required column: Class" }
```

3.3 Error Handling

HTTP error taxonomy. Every synchronous error maps to an HTTP status, a machine-readable code and a safe client message (never internals, per SR-06; never cell values, per SR-04):

Code	HTTP	Cause	Client message
E400_VALIDATION	400	Unsupported file type, size over limit, wrong delimiter, schema or label violation, malformed or unknown JSON fields, invalid model_type	Names the reason, e.g. "missing required column: Class"
E401_AUTH	401	Missing or invalid API key	"Authentication required: provide a valid API key."
E404_NOT_FOUND	404	Unknown or expired dataset_id or job_id	"The requested dataset/job was not found."
E405_METHOD	405	HTTP method not supported on the route	"Method not allowed for this endpoint."
E500_INTERNAL	500	Unexpected Web Server failure	"An internal error occurred." (details logged server-side only)
E500_MODEL	500	Model failure inside a microservice (POST /predict only)	"The model failed to process the dataset."

Job failure reasons. Microservice problems happen after HTTP 202 has been returned, so they are not HTTP errors to the client. They are recorded in the job record and reported by GET /api/analyses/{job_id} (HTTP 200, status "failed"):

Reason code	Cause	Message
MODEL_SERVICE_ERROR	Microservice unreachable, returned 4xx/5xx, or returned a body that does not match the IPredict contract	"The model service is unavailable; the job was marked failed."
JOB_TIMEOUT	No response within 600 s of the job entering running	"The analysis timed out; the job was marked failed."

Job lifecycle.

From	To	Trigger
—	queued	POST /api/analyses accepted (job record written, HTTP 202 returned)
queued	running	Background worker picks up the job and calls POST /predict
running	completed	Microservice returns 200 with a valid body; result written
running	failed	MODEL_SERVICE_ERROR or JOB_TIMEOUT

completed and failed are terminal: a job never leaves them, and it is deleted with its dataset at the end of the retention period.

Policies. (1) Validation errors are detected before any job creation or model invocation. (2) Microservice failures and timeouts mark the job failed with a recorded reason; a failed job is never presented as successful (REQ-FD-14, BR-04). (3) The client polls until a terminal state; on failed it displays the reason and lets the user start a new run. (4) All unexpected exceptions are caught by the global error handler, logged server-side without transaction data (SR-04), and returned as E500_INTERNAL.

3.4 Module Breakdown

Planned structure of the implementation.

Flask Web Server (server/):

- `app.py` — Flask application factory, configuration loading (API key, store path, service addresses, timeouts), route registration, static file serving, global error handler.
- `routes.py` — handlers for POST /api/datasets, POST /api/analyses and GET /api/analyses/{job_id}.
- `auth.py` — API-Key Auth Filter: validates the Authorization header on every /api route.
- `validator.py` — Input Validator: file-level and row-level CSV checks; JSON schema validation.
- `jobs.py` — Job Manager: job creation, background worker pool, POST /predict call with the 600 s timeout, recording of completed/failed states and reasons.
- `store.py` — Store Access: save/read/delete datasets, job records and results; owner-only permissions; retention purge task.

Each model microservice (services/ml, services/dl, services/qml):

- `server.py` — HTTP layer exposing POST /predict and GET /health, bound to 127.0.0.1.
- `dataset_reader.py` — opens `datasets/<dataset_id>.csv` from the shared store (read-only) and selects the model's feature columns.
- `model_loader.py` — loads the trained classifier named in the service configuration at startup.
- `predictor.py` — runs inference over the dataset rows and emits per-row class and probability.

- `metrics.py` — computes accuracy, precision, recall, F1, ROC-AUC and the confusion matrix against the Class column.

Web Client (client/):

- `index.html` — the four regions: upload (with API-key field), model selection, run control/status, results.
- `style.css` — the single shared stylesheet (REQ-UI-06).
- `app.js` — API-key handling (memory only), upload submission, run control, 2-second polling loop, results rendering with 50-row pagination, CSV export built in the browser, error display, help link.

3.5 User Interface Design Notes

- Single-page layout with four regions (REQ-UI-01): dataset upload (API-key field, single-file chooser, format hint), model selection (ML/DL/QML radio options, exactly one required — REQ-FD-06), run control (start button, live status badge: queued/running/completed/failed — REQ-UI-03), results (predictions table, metrics panel, confusion matrix, model name, download button).
- The API key typed by the user is held in a JavaScript variable only; it is not written to cookies or browser storage and must be re-entered after a page reload (SR-03).
- Error messages appear adjacent to the control that caused them within 1 s of the server response (REQ-UI-02).
- When a job finishes, the results (or the failure reason) replace the status view automatically (REQ-FD-20).
- The analytical-only notice ("Analytical result only — not a financial authorization decision.") is permanently visible in the results region (NFR-S02).
- A help link to the user manual is visible on every screen (REQ-UI-04); one shared stylesheet keeps styling consistent and the layout targets viewports of 1280×720 and above (REQ-UI-06).

Appendix A: Glossary

Term	Meaning
SADS	Software Architecture & Design Specification (this document, SADS-CCFD-1.1).
Provided interface	An interface a component offers to others (ball notation), e.g. IRestApi, IPredict.
Required interface	An interface a component needs from another (socket notation).
IRestApi / IPredict / IDatasetStore	The three interfaces of the system: Web Server REST API, microservice contract, and dataset store file access.
«use» dependency	A dashed UML dependency: the source component requires the target's interface.
Combined fragment	A UML sequence-diagram region (alt, par, loop) grouping alternative, parallel or repeated interactions.
Job	An asynchronous analysis run tracked through queued → running → completed/failed.
Retention period	24 hours from upload, after which a dataset and its jobs and results are deleted.

Project Team

SRN	Name
PES1UG24AM024	Akshat Agarwal
PES1UG24AM030	Aman Priyansh
PES1UG24AM015	Aditya Kalloli
PES1UG23AM224	Raghav Balaji